AI-Powered Natural Language Search — Feature Roadmap

> ***Platform:*** *MATA Matrimony* ***API:*** `matrimonyservicesapi` *(Node.js / TypeScript / Express)* ***Database:*** *MySQL (stored procedures + parameterized queries)* ***Feature Type:*** *Premium Service* ***Status:*** *Architecture Approved — Pending Implementation* ***Last Updated:*** *March 2026*

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Overview
4. Architecture
   - 4.1 High-Level Flow
   - 4.2 Design Philosophy
   - 4.3 Component Diagram
5. Current State Analysis
   - 5.1 Existing Search API
   - 5.2 Database Tables
   - 5.3 Lookup Table — The Bridge
6. AI Provider Abstraction Layer
   - 6.1 Provider Interface
   - 6.2 Search Intent Schema
   - 6.3 Provider Factory
   - 6.4 Provider Comparison
   - 6.5 Environment Configuration
7. Core Components
   - 7.1 Schema Context Builder
   - 7.2 Lookup Resolver
   - 7.3 Intent Validator
   - 7.4 Query Builder
   - 7.5 AI Search Service (Orchestrator)
8. API Endpoints
   - 8.1 AI Search
   - 8.2 Search History
   - 8.3 Search Refinement
   - 8.4 Cache Management
9. Database Changes
   - 9.1 New Tables
   - 9.2 New Stored Procedures
10. Security & Safety
11. Premium Gating & Billing
12. File Structure
13. Implementation Phases
14. End-to-End Example
15. Error Handling
16. Performance Considerations

17. Monitoring & Observability
18. Future Enhancements
19. Appendix

---

1. Executive Summary

This document describes the architecture and implementation plan for an **AI-powered natural language search** feature for the MATA Matrimony platform. Instead of requiring users to manually select filters from dropdowns, users type what they are looking for in plain English, and the system uses an AI language model to interpret the query, extract structured filters, resolve them against the database, and return matching profiles.

Key characteristics:

- **Natural language input** — "Find Hindu women between 25-30, MBA, from Hyderabad, vegetarian"
- **AI-agnostic** — works with OpenAI, Google Gemini, Anthropic Claude, or DeepSeek (swappable via env config)
- **Secure by design** — AI never generates SQL; it outputs structured JSON that the API validates and converts to parameterized queries
- **Premium service** — gated by subscription tier with quota enforcement
- **Fully auditable** — every search is logged with query, interpretation, filters, results count, tokens used, and latency

---

2. Problem Statement

Current Limitations

The existing search endpoint (`POST /api/profile/search`) requires users to:

1. Know which filters are available (age, religion, education, etc.)
2. Select exact values from dropdown lists
3. Manually combine multiple filter criteria
4. Cannot express complex or nuanced preferences

User Expectations

Modern users expect to describe what they want in natural language:

- *"I want a tall, well-educated girl from a good family in Mumbai"*
- *"Looking for a software engineer, 28-32, who likes travelling and is a vegetarian"*
- *"Find Brahmin boys with MBA working in Bangalore, non-smoker"*

Business Opportunity

Natural language search can be offered as a **premium feature**, creating a new revenue stream while significantly improving user experience and engagement.

---

3. Solution Overview

```
User types natural language
        │
        ▼
AI converts to structured JSON filters
```

```
        │
        ▼
API validates, resolves lookups, builds safe SQL
        │
        ▼
MySQL returns matching profiles
        │
        ▼
Response includes profiles + AI interpretation
```

The AI acts as a **query planner**, not a query executor. It translates human intent into machine-readable filters. The API layer handles all database interaction with full security controls.

---

4. Architecture

4.1 High-Level Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                        USER (Frontend)                           │
│  "Find Hindu women between 25-30, MBA, working in IT,           │
│   living in Hyderabad, non-smoker, vegetarian"                   │
└──────────────────┬──────────────────────────────────────────────┘
                   │  POST /api/ai-search
                   ▼
┌──────────────────────────────────────────────────────────────────┐
│                   AI SEARCH CONTROLLER                           │
│                                                                  │
│  1. Validate request (auth, API key, input sanitization)         │
│  2. Check premium subscription status                            │
│  3. Check & enforce search quota                                 │
│  4. Load schema context + lookup values from memory cache        │
│  5. Send user query + schema context to AI Provider              │
│  6. Receive structured JSON intent from AI                       │
│  7. Validate AI output against JSON schema                       │
│  8. Resolve lookup text names → numeric IDs via lookup_table     │
│  9. Build parameterized SQL query (safe WHERE clause)            │
│ 10. Execute query via MySQL connection pool                      │
│ 11. Log search to ai_search_log table                            │
│ 12. Increment quota usage                                        │
│ 13. Return profiles + interpretation + applied filters           │
└──────────────────────────────────────────────────────────────────┘
```

4.2 Design Philosophy

"AI as Query Planner, Not Query Executor"

Principle	Description
AI never touches the database	All DB operations go through the API's parameterized query builder

AI outputs JSON, not SQL	Structured output is testable, validatable, and safe
Provider-agnostic	Strategy pattern allows swapping AI providers via env config
Lookup cache in memory	Avoids DB round-trip per request; lookup data is near-static
Every search is logged	Enables billing, debugging, prompt optimization, and auditing
Fail-safe defaults	If AI returns garbage, the system returns an error — never executes bad queries

Why NOT "AI generates raw SQL"?

Risk	Impact
SQL injection	LLM could be prompt-injected to produce malicious SQL
Schema exposure	Sending full DDL to AI leaks internal database structure
Non-deterministic	Same prompt may produce different SQL across calls
Untestable	Cannot unit-test raw SQL generation reliably
No audit trail	Harder to log and debug free-form SQL

4.3 Component Diagram

```
┌──────────────────────────────────────────────────────────┐
│                    API Layer                             │
│                                                          │
│  ┌──────────────┐    ┌──────────────┐                   │
│  │  aiSearch    │    │  premiumCheck  │                 │
│  │  .routes.ts  │──▶│  .middleware   │                  │
│  └──────────────┘    └──────┬───────┘                   │
│                             │                            │
│                    ┌────────▼────────┐                  │
│                    │    aiSearch      │                 │
│                    │    .controller.ts  │               │
│                    └──────┬──────────┘                  │
│                           │                              │
│                    ┌────────▼────────┐                  │
│                    │    aiSearch      │                 │
│                    │    .service.ts    │  ◀── Orchestrator
│                    └──┬──┬──┬──┬──┬──┘                  │
│                       │  │  │  │                        │
│         ┌─────────────┘  │  │  └──────────────┐         │
│         ▼                ▼  ▼                 ▼         │
│  ┌──────────┐  ┌──────────┐ ┌──────────┐  ┌──────────┐ │
│  │ schema   │  │ lookup   │ │ intent   │  │ query    │ │
│  │ Context  │  │ Resolver │ │ Validator│  │ Builder  │ │
│  └──────┬───┘  └──────────┘ └──────────┘  └──────┬───┘ │
│       │                                          │       │
│       ▼                                          ▼       │
```

```
│  ┌──────────┐                          ┌──────────────┐ │
│  │ AI       │                          │  MySQL Pool  │ │
│  │ Provider │                          │  (database)  │ │
│  │ Factory  │                          └──────────────┘ │
│  └────┬─────┘                                           │
│       │                                                 │
│  ┌────▼──────────────────────────────┐                  │
│  │  Provider Implementations         │                  │
│  │  ┌────────┐ ┌────────┐ ┌───────┐ │                  │
│  │  │ OpenAI │ │ Gemini │ │Claude │ │                  │
│  │  └────────┘ └────────┘ └───────┘ │                  │
│  │  ┌──────────┐                    │                  │
│  │  │ DeepSeek │                    │                  │
│  │  └──────────┘                    │                  │
│  └──────────────────────────────────┘                  │
└────────────────────────────────────────────────────────┘
```

---

5. Current State Analysis

5.1 Existing Search API

Aspect	Details
Endpoint	`POST /api/profile/search`
Stored Procedure	`eb_profile_search_get(profile_id, min_age, max_age, religion, max_education, occupation, country, caste_id, marital_status, gender)`
Filter Count	10 fixed parameters
Auth	JWT + API Key
Limitations	Rigid filters, dropdown-only, no free-text, no combined/complex queries

Additional search-related endpoints:

Endpoint	Purpose
`POST /api/profile/search/preferences`	Save user's filter preferences
`GET /api/profile/search/preferences/:profileId`	Retrieve saved preferences
`POST /api/profile/allProfiles`	Browse all active profiles

5.2 Database Tables

All profile data is spread across normalized tables. The AI search must be able to query across all of them.

Table	Key Searchable Columns	Stored Procedure Pattern
`profile_personal`	first_name, last_name, gender, birth_date, marital_status, religion, caste, nationality,	`eb_profile_personal_*`

	height_inches, height_cms, weight, complexion, profession, short_summary	
`profile_address`	address_line1, city, state, country, zip	`eb_profile_address_*`
`profile_education`	education_level, institution_name, field_of_study, year_completed	`eb_profile_education_*`
`profile_employment`	institution_name (company), job_title_id, last_salary_drawn, start_year, end_year	`eb_profile_employment_*`
`profile_property`	property_type, ownership_type, property_value, property_description	`eb_profile_property_*`
`profile_family_reference`	reference_type, first_name, last_name, highest_education, marital_status, employment_status	`eb_profile_family_reference_*`
`profile_lifestyle`	eating_habit, diet_habit, drink_frequency, cigarettes_per_day, gambling_engage, physical_activity_level	`eb_profile_lifestyle_*`
`profile_hobby_interest`	hobby (lookup ID)	`eb_profile_hobby_interest_*`
`profile_photo`	url, photo_type, caption, relative_path	`eb_profile_photo_*`
`lookup_table`	id, name, description, category, isactive	`lkp_get_LookupData(category)`

5.3 Lookup Table — The Bridge

Critical design constraint: Most filter columns (gender, religion, caste, marital_status, education_level, etc.) store **numeric IDs** that reference the `lookup_table`, not text strings.

Example:

User says	DB column	lookup_table entry
"Hindu"	`religion = 3`	`{id: 3, name: "Hindu", category: "religion"}`
"MBA"	`education_level = 12`	`{id: 12, name: "MBA", category: "education_level"}`
"Female"	`gender = 2`	`{id: 2, name: "Female", category: "gender"}`

The AI search system must:

1. Know all valid lookup values (loaded into the prompt)
2. Receive text from the AI (e.g., "Hindu")
3. Resolve text → numeric ID via the lookup cache
4. Use the numeric ID in the parameterized SQL query

---

6. AI Provider Abstraction Layer

6.1 Provider Interface

Every AI provider must implement this interface:

```typescript
// src/services/aiSearch/providers/aiProvider.interface.ts

export interface IAIProvider {
  /** Provider name for logging */
  name: string;

  /** Model identifier */
  model: string;

  /**
   * Parse a natural language search query into structured filters.
   * @param userQuery - The user's natural language input
   * @param schemaContext - The prompt context with available filters and lookup values
   * @returns Structured search intent
   */
  parseSearchIntent(
    userQuery: string,
    schemaContext: string
  ): Promise<IAISearchResponse>;
}

export interface IAISearchResponse {
  intent: ISearchIntent;
  tokensUsed: {
    prompt: number;
    completion: number;
    total: number;
  };
  responseTimeMs: number;
}
```

6.2 Search Intent Schema

This is the structured JSON that every AI provider must return:

```typescript
// src/interfaces/aiSearch.interface.ts

export interface ISearchIntent {
  filters: ISearchFilters;
  sort_by?: 'age_asc' | 'age_desc' | 'newest' | 'salary_desc' | 'height_desc';
  confidence: number;       // 0.0 to 1.0
  interpretation: string;   // Human-readable explanation of what the AI understood
}

export interface ISearchFilters {
  // Personal
  gender?: string;
  min_age?: number;
  max_age?: number;
  religion?: string;
  caste?: string;
```

```
  marital_status?: string;
  nationality?: string;
  min_height_cms?: number;
  max_height_cms?: number;
  min_weight?: number;
  max_weight?: number;
  complexion?: string;

  // Location
  city?: string;
  state?: string;
  country?: string;

  // Education
  education_level?: string;
  field_of_study?: string;
  institution_name?: string;

  // Employment
  occupation?: string;         // job title
  company?: string;            // employer name
  min_salary?: number;
  max_salary?: number;

  // Lifestyle
  eating_habit?: string;
  drinking?: string;
  smoking?: string;
  physical_activity?: string;

  // Other
  hobbies?: string[];
  property_owner?: boolean;
  disability?: string;
}

export interface IResolvedIds {
  gender?: number | null;
  religion?: number | null;
  caste?: number | null;
  marital_status?: number | null;
  nationality?: number | null;
  complexion?: number | null;
  education_level?: number | null;
  field_of_study?: number | null;
  occupation?: number | null;
  eating_habit?: number | null;
  drinking?: number | null;
  smoking?: number | null;
  physical_activity?: number | null;
  hobbies?: (number | null)[];
  disability?: number | null;
```

```
}

export interface IAISearchRequest {
  query: string;
  profile_id: number;
}

export interface IAISearchResult {
  success: boolean;
  data?: {
    profiles: any[];
    interpretation: string;
    filters_applied: ISearchFilters;
    resolved_ids: IResolvedIds;
    confidence: number;
    result_count: number;
    ai_provider: string;
    ai_model: string;
    tokens_used: number;
    response_time_ms: number;
  };
  message?: string;
  error?: string;
}
```

6.3 Provider Factory

```
// src/services/aiSearch/aiProviderFactory.ts

import { IAIProvider } from './providers/aiProvider.interface';
import { OpenAIProvider } from './providers/openai.provider';
import { GeminiProvider } from './providers/gemini.provider';
import { ClaudeProvider } from './providers/claude.provider';
import { DeepSeekProvider } from './providers/deepseek.provider';

export function createAIProvider(): IAIProvider {
  const provider = (process.env.AI_PROVIDER || 'openai').toLowerCase();

  switch (provider) {
    case 'openai':
      return new OpenAIProvider();
    case 'gemini':
      return new GeminiProvider();
    case 'claude':
      return new ClaudeProvider();
    case 'deepseek':
      return new DeepSeekProvider();
    default:
      throw new Error(
        `Unknown AI provider: "${provider}". ` +
        `Supported: openai, gemini, claude, deepseek`
```

```
      );
  }
}
```

6.4 Provider Comparison

Provider	Recommended Model	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)	Avg Latency	JSON Quality	Notes
OpenAI	`gpt-4o-mini`	~$0.15	~$0.60	1–2s	Excellent	**Recommended default** — best balance of cost, speed, and structured output
Google Gemini	`gemini-2.0-flash`	~$0.10	~$0.40	1–2s	Very Good	Cheapest input tokens; good alternative
DeepSeek	`deepseek-chat`	~$0.14	~$0.28	2–3s	Good	Cheapest output tokens; slightly slower
Anthropic Claude	`claude-3.5-haiku`	~$0.25	~$1.25	1–2s	Excellent	Best instruction-following; most expensive

Estimated cost per AI search request:

- Prompt: ~800 tokens (schema context + user query)
- Completion: ~200 tokens (structured JSON response)
- **Cost per search: ~$0.0002 with gpt-4o-mini** (≈ 5,000 searches per $1)

6.5 Environment Configuration

```
# ─── AI Search Configuration ──────────────────────────
# Provider: openai | gemini | claude | deepseek
AI_PROVIDER=openai

# API key for the selected provider
AI_API_KEY=sk-...

# Model name (provider-specific)
AI_MODEL=gpt-4o-mini

# Max tokens for AI response (keep low for cost control)
AI_MAX_TOKENS=500

# Temperature: 0.0-1.0 (lower = more deterministic)
AI_TEMPERATURE=0.1
```

```
# Timeout in milliseconds for AI API calls
AI_TIMEOUT_MS=15000

# ─── AI Search Quotas ──────────────────────────────────
# Default monthly search limit for free users
AI_SEARCH_FREE_LIMIT=5

# Default monthly search limit for basic plan
AI_SEARCH_BASIC_LIMIT=50

# Default monthly search limit for premium plan
AI_SEARCH_PREMIUM_LIMIT=500
```

---

7. Core Components

7.1 Schema Context Builder

The Schema Context Builder is responsible for:

1. Loading all lookup table values into memory at startup
2. Building the AI system prompt with available filter options and their valid values
3. Providing a refresh mechanism for when lookup data changes

```
// src/services/aiSearch/schemaContext.ts

export class SchemaContext {
  private lookupCache: Map<string, { id: number; name: string }[]> = new Map();
  private systemPrompt: string = '';
  private lastRefreshed: Date | null = null;

  /**
   * Initialize lookup cache from database.
   * Called once at server startup and on-demand via admin endpoint.
   */
  async initialize(): Promise<void> {
    const categories = [
      'gender', 'religion', 'caste', 'marital_status',
      'education_level', 'field_of_study', 'job_title',
      'complexion', 'eating_habit', 'smoking', 'drinking',
      'physical_activity', 'nationality', 'disability',
      'hobby', 'property_type', 'ownership_type'
    ];

    for (const category of categories) {
      const [rows] = await pool.execute(
        'CALL lkp_get_LookupData(?)',
        [category]
      );
      this.lookupCache.set(category, (rows as any[])[0] || []);
    }
```

```
    // Also load countries and states
    const [countries] = await pool.execute('CALL lkp_get_Country_List()');
    this.lookupCache.set('country', (countries as any[])[0] || []);

    this.systemPrompt = this.buildSystemPrompt();
    this.lastRefreshed = new Date();
  }

  /**
   * Build the system prompt that gives the AI knowledge of
   * available filters and valid values.
   */
  private buildSystemPrompt(): string {
    return `You are a matrimony profile search assistant for MATA Matrimony.
Your ONLY job is to parse natural language search queries into structured JSON filters.

AVAILABLE FILTERS AND VALID VALUES:

1. PERSONAL FILTERS:
   - gender: ${this.getLookupNames('gender')}
   - religion: ${this.getLookupNames('religion')}
   - caste: ${this.getLookupNames('caste')}
   - marital_status: ${this.getLookupNames('marital_status')}
   - nationality: ${this.getLookupNames('nationality')}
   - complexion: ${this.getLookupNames('complexion')}
   - disability: ${this.getLookupNames('disability')}
   - min_age, max_age: integers between 18 and 80
   - min_height_cms, max_height_cms: integers (e.g., 150 to 200)
   - min_weight, max_weight: integers in kg

2. LOCATION FILTERS:
   - city: free text (e.g., "Hyderabad", "Mumbai", "Chennai")
   - state: free text (e.g., "Telangana", "Maharashtra")
   - country: ${this.getLookupNames('country')}

3. EDUCATION FILTERS:
   - education_level: ${this.getLookupNames('education_level')}
   - field_of_study: ${this.getLookupNames('field_of_study')}
   - institution_name: free text

4. EMPLOYMENT FILTERS:
   - occupation: ${this.getLookupNames('job_title')}
   - company: free text
   - min_salary, max_salary: integers (annual income)

5. LIFESTYLE FILTERS:
   - eating_habit: ${this.getLookupNames('eating_habit')}
   - drinking: ${this.getLookupNames('drinking')}
   - smoking: ${this.getLookupNames('smoking')}
   - physical_activity: ${this.getLookupNames('physical_activity')}
```

```
6. OTHER FILTERS:
   - hobbies: array of strings from ${this.getLookupNames('hobby')}
   - property_owner: true or false

7. SORTING (optional):
   - sort_by: "age_asc", "age_desc", "newest", "salary_desc", "height_desc"

RESPONSE FORMAT — Return ONLY valid JSON:
{
  "filters": { /* only include filters mentioned by user */ },
  "sort_by": "newest",
  "confidence": 0.85,
  "interpretation": "Brief human-readable summary of what you understood"
}

RULES:
1. Return ONLY valid JSON — no markdown, no code blocks, no explanation outside the JSON.
2. Only include filters that the user explicitly or clearly implies.
3. Set confidence between 0.0 and 1.0 based on clarity of the query.
4. If the query is vague, include fewer filters and lower confidence.
5. If the query is unrelated to profile search, return: {"filters": {}, "confidence": 0,
"interpretation": "Query not related to profile search"}
6. Use the closest matching value from the valid options listed above.
7. When the user mentions "tall", interpret as min_height_cms: 170.
8. When the user mentions "well-educated", interpret as education_level: "Post Graduate" or
higher.
9. When the user mentions "good salary" or "well-earning", interpret as min_salary: 1000000.
10. Always infer gender from context if not explicitly stated (e.g., "bride" = Female,
"groom" = Male).
11. Never include SQL, code, or anything outside the JSON schema.`;
  }

  private getLookupNames(category: string): string {
    const items = this.lookupCache.get(category) || [];
    const names = items.map(item => `"${item.name}"`).join(', ');
    return names || '"(none loaded)"';
  }

  getSystemPrompt(): string {
    return this.systemPrompt;
  }

  getLookupCache(): Map<string, { id: number; name: string }[]> {
    return this.lookupCache;
  }

  getLastRefreshed(): Date | null {
    return this.lastRefreshed;
  }
}
```

7.2 Lookup Resolver

Converts text values from the AI response to numeric IDs using the lookup cache.

```
// src/services/aiSearch/lookupResolver.ts

export class LookupResolver {
  /**
   * Resolve all text-based filter values to their numeric lookup IDs.
   * Uses exact match first, then case-insensitive, then substring match.
   */
  resolve(
    filters: ISearchFilters,
    lookupCache: Map<string, { id: number; name: string }[]>
  ): IResolvedIds {
    const resolved: IResolvedIds = {};

    // Map of filter field → lookup category
    const fieldCategoryMap: Record<string, string> = {
      gender: 'gender',
      religion: 'religion',
      caste: 'caste',
      marital_status: 'marital_status',
      nationality: 'nationality',
      complexion: 'complexion',
      education_level: 'education_level',
      field_of_study: 'field_of_study',
      occupation: 'job_title',
      eating_habit: 'eating_habit',
      drinking: 'drinking',
      smoking: 'smoking',
      physical_activity: 'physical_activity',
      disability: 'disability',
    };

    for (const [field, category] of Object.entries(fieldCategoryMap)) {
      const value = (filters as any)[field];
      if (value && typeof value === 'string') {
        const lookups = lookupCache.get(category) || [];
        (resolved as any)[field] = this.fuzzyMatch(value, lookups);
      }
    }

    // Resolve hobbies array
    if (filters.hobbies && filters.hobbies.length > 0) {
      const hobbyLookups = lookupCache.get('hobby') || [];
      resolved.hobbies = filters.hobbies.map(h =>
        this.fuzzyMatch(h, hobbyLookups)
      );
    }

    return resolved;
```

```
  }

  /**
   * Match a text value against lookup entries.
   * Priority: exact → case-insensitive → contains → partial
   */
  private fuzzyMatch(
    text: string,
    lookups: { id: number; name: string }[]
  ): number | null {
    if (!text || !lookups.length) return null;

    const lower = text.toLowerCase().trim();

    // 1. Exact match
    const exact = lookups.find(l => l.name === text);
    if (exact) return exact.id;

    // 2. Case-insensitive match
    const caseInsensitive = lookups.find(
      l => l.name.toLowerCase() === lower
    );
    if (caseInsensitive) return caseInsensitive.id;

    // 3. Lookup name contains search text
    const contains = lookups.find(
      l => l.name.toLowerCase().includes(lower)
    );
    if (contains) return contains.id;

    // 4. Search text contains lookup name
    const partial = lookups.find(
      l => lower.includes(l.name.toLowerCase())
    );
    if (partial) return partial.id;

    return null;
  }
}
```

7.3 Intent Validator

Validates the AI's JSON response against the expected schema.

```
// src/services/aiSearch/intentValidator.ts

export class IntentValidator {
  /**
   * Validate and sanitize the AI's raw JSON output.
   * Returns a clean ISearchIntent or throws on invalid input.
   */
```

```
validate(raw: any): ISearchIntent {
  // Must be an object
  if (!raw || typeof raw !== 'object') {
    throw new Error('AI returned non-object response');
  }

  // Must have filters object
  if (!raw.filters || typeof raw.filters !== 'object') {
    throw new Error('AI response missing "filters" object');
  }

  // Sanitize confidence
  const confidence = typeof raw.confidence === 'number'
    ? Math.min(1, Math.max(0, raw.confidence))
    : 0.5;

  // Sanitize interpretation
  const interpretation = typeof raw.interpretation === 'string'
    ? raw.interpretation.substring(0, 500)
    : 'No interpretation provided';

  // Validate individual filter types
  const filters = this.sanitizeFilters(raw.filters);

  // Validate sort_by
  const validSorts = ['age_asc', 'age_desc', 'newest', 'salary_desc', 'height_desc'];
  const sort_by = validSorts.includes(raw.sort_by) ? raw.sort_by : undefined;

  return { filters, sort_by, confidence, interpretation };
}

private sanitizeFilters(raw: any): ISearchFilters {
  const clean: ISearchFilters = {};

  // String fields — only accept non-empty strings, max 100 chars
  const stringFields = [
    'gender', 'religion', 'caste', 'marital_status', 'nationality',
    'complexion', 'city', 'state', 'country', 'education_level',
    'field_of_study', 'institution_name', 'occupation', 'company',
    'eating_habit', 'drinking', 'smoking', 'physical_activity', 'disability'
  ];
  for (const field of stringFields) {
    if (raw[field] && typeof raw[field] === 'string') {
      (clean as any)[field] = raw[field].trim().substring(0, 100);
    }
  }

  // Numeric fields — only accept positive integers within range
  const numericFields: [string, number, number][] = [
    ['min_age', 18, 80], ['max_age', 18, 80],
    ['min_height_cms', 100, 250], ['max_height_cms', 100, 250],
    ['min_weight', 30, 200], ['max_weight', 30, 200],
```

```
      ['min_salary', 0, 100000000], ['max_salary', 0, 100000000],
    ];
    for (const [field, min, max] of numericFields) {
      const val = raw[field];
      if (typeof val === 'number' && val >= min && val <= max) {
        (clean as any)[field] = Math.round(val);
      }
    }

    // Boolean fields
    if (typeof raw.property_owner === 'boolean') {
      clean.property_owner = raw.property_owner;
    }

    // Array fields — hobbies
    if (Array.isArray(raw.hobbies)) {
      clean.hobbies = raw.hobbies
        .filter((h: any) => typeof h === 'string' && h.trim())
        .map((h: string) => h.trim().substring(0, 100))
        .slice(0, 10); // max 10 hobbies
    }

    return clean;
  }
}
```

7.4 Query Builder

Converts validated search intent + resolved IDs into a safe parameterized SQL query.

```
// src/services/aiSearch/queryBuilder.ts

export class QueryBuilder {
  /**
   * Build a parameterized SQL query from validated filters.
   * EVERY value goes through ? placeholder — zero SQL injection risk.
   */
  build(
    intent: ISearchIntent,
    resolvedIds: IResolvedIds,
    callerProfileId: number
  ): { sql: string; params: any[] } {
    const conditions: string[] = [
      'pp.softdelete = 0',
      'pp.profile_id != ?'
    ];
    const params: any[] = [callerProfileId];
    const joins: Set<string> = new Set();

    // ── Personal filters ──────────────────────────
    if (resolvedIds.gender != null) {
```

```
      conditions.push('pp.gender = ?');
      params.push(resolvedIds.gender);
    }

    if (intent.filters.min_age != null) {
      conditions.push('TIMESTAMPDIFF(YEAR, pp.birth_date, CURDATE()) >= ?');
      params.push(intent.filters.min_age);
    }

    if (intent.filters.max_age != null) {
      conditions.push('TIMESTAMPDIFF(YEAR, pp.birth_date, CURDATE()) <= ?');
      params.push(intent.filters.max_age);
    }

    if (resolvedIds.religion != null) {
      conditions.push('pp.religion = ?');
      params.push(resolvedIds.religion);
    }

    if (resolvedIds.caste != null) {
      conditions.push('pp.caste = ?');
      params.push(resolvedIds.caste);
    }

    if (resolvedIds.marital_status != null) {
      conditions.push('pp.marital_status = ?');
      params.push(resolvedIds.marital_status);
    }

    if (resolvedIds.complexion != null) {
      conditions.push('pp.complexion = ?');
      params.push(resolvedIds.complexion);
    }

    if (intent.filters.min_height_cms != null) {
      conditions.push('pp.height_cms >= ?');
      params.push(intent.filters.min_height_cms);
    }

    if (intent.filters.max_height_cms != null) {
      conditions.push('pp.height_cms <= ?');
      params.push(intent.filters.max_height_cms);
    }

    // ── Location filters (require address join) ────────
    if (intent.filters.city) {
      joins.add('LEFT JOIN profile_address pa ON pp.profile_id = pa.profile_id AND
pa.softdelete = 0');
      conditions.push('pa.city LIKE ?');
      params.push(`%${intent.filters.city}%`);
    }
```

```
    if (intent.filters.state) {
      joins.add('LEFT JOIN profile_address pa ON pp.profile_id = pa.profile_id AND
pa.softdelete = 0');
      conditions.push('pa.state LIKE ?');
      params.push(`%${intent.filters.state}%`);
    }

    if (intent.filters.country) {
      joins.add('LEFT JOIN profile_address pa ON pp.profile_id = pa.profile_id AND
pa.softdelete = 0');
      conditions.push('pa.country LIKE ?');
      params.push(`%${intent.filters.country}%`);
    }

    // ── Education filters (require education join) ────
    if (resolvedIds.education_level != null) {
      joins.add('LEFT JOIN profile_education pe ON pp.profile_id = pe.profile_id AND
pe.softdelete = 0');
      conditions.push('pe.education_level = ?');
      params.push(resolvedIds.education_level);
    }

    if (resolvedIds.field_of_study != null) {
      joins.add('LEFT JOIN profile_education pe ON pp.profile_id = pe.profile_id AND
pe.softdelete = 0');
      conditions.push('pe.field_of_study = ?');
      params.push(resolvedIds.field_of_study);
    }

    // ── Employment filters (require employment join) ───
    if (resolvedIds.occupation != null) {
      joins.add('LEFT JOIN profile_employment pem ON pp.profile_id = pem.profile_id AND
pem.softdelete = 0');
      conditions.push('pem.job_title_id = ?');
      params.push(resolvedIds.occupation);
    }

    if (intent.filters.min_salary != null) {
      joins.add('LEFT JOIN profile_employment pem ON pp.profile_id = pem.profile_id AND
pem.softdelete = 0');
      conditions.push('CAST(pem.last_salary_drawn AS UNSIGNED) >= ?');
      params.push(intent.filters.min_salary);
    }

    if (intent.filters.max_salary != null) {
      joins.add('LEFT JOIN profile_employment pem ON pp.profile_id = pem.profile_id AND
pem.softdelete = 0');
      conditions.push('CAST(pem.last_salary_drawn AS UNSIGNED) <= ?');
      params.push(intent.filters.max_salary);
    }

    // ── Lifestyle filters (require lifestyle join) ────
```

```
    const lifestyleNeeded = resolvedIds.eating_habit != null
      || resolvedIds.drinking != null
      || resolvedIds.smoking != null
      || resolvedIds.physical_activity != null;

    if (lifestyleNeeded) {
      joins.add('LEFT JOIN profile_lifestyle pl ON pp.profile_id = pl.profile_id AND
pl.softdelete = 0');
    }

    if (resolvedIds.eating_habit != null) {
      conditions.push('pl.eating_habit = ?');
      params.push(resolvedIds.eating_habit);
    }

    if (resolvedIds.drinking != null) {
      conditions.push('pl.drink_frequency = ?');
      params.push(resolvedIds.drinking);
    }

    if (resolvedIds.smoking != null) {
      conditions.push('pl.cigarettes_per_day = ?');
      params.push(resolvedIds.smoking);
    }

    // ── Photo join (always, for primary photo) ──────────
    const photoJoin = 'LEFT JOIN profile_photo ph ON pp.profile_id = ph.profile_id AND
ph.photo_type = 450 AND ph.softdelete = 0';

    // ── Build ORDER BY ──────────────────────────────────
    let orderBy = 'pp.created_date DESC';
    switch (intent.sort_by) {
      case 'age_asc':     orderBy = 'pp.birth_date DESC'; break;
      case 'age_desc':    orderBy = 'pp.birth_date ASC'; break;
      case 'newest':      orderBy = 'pp.created_date DESC'; break;
      case 'height_desc': orderBy = 'pp.height_cms DESC'; break;
    }

    // ── Assemble final SQL ──────────────────────────────
    const allJoins = [...joins, photoJoin].join('\n      ');

    const sql = `
      SELECT DISTINCT
        pp.profile_id,
        pp.first_name,
        pp.last_name,
        pp.gender,
        pp.birth_date,
        TIMESTAMPDIFF(YEAR, pp.birth_date, CURDATE()) AS age,
        pp.religion,
        pp.caste,
        pp.marital_status,
```

```
        pp.nationality,
        pp.height_cms,
        pp.weight,
        pp.complexion,
        pp.profession,
        pp.short_summary,
        ph.url AS primary_photo_url,
        ph.relative_path AS primary_photo_path
      FROM profile_personal pp
      ${allJoins}
      WHERE ${conditions.join('\n        AND ')}
      ORDER BY ${orderBy}
      LIMIT 50
    `;

    return { sql, params };
  }
}
```

Safety guarantees:

- Every value uses `?` parameterized placeholder
- Column names are hardcoded (not from AI)
- JOINs are from a fixed set
- LIMIT is always enforced
- AI output never appears in SQL — only resolved numeric IDs and sanitized text

7.5 AI Search Service (Orchestrator)

```
// src/services/aiSearch/aiSearch.service.ts

export class AISearchService {
  private provider: IAIProvider;
  private schemaContext: SchemaContext;
  private lookupResolver: LookupResolver;
  private intentValidator: IntentValidator;
  private queryBuilder: QueryBuilder;

  constructor() {
    this.provider = createAIProvider();
    this.schemaContext = new SchemaContext();
    this.lookupResolver = new LookupResolver();
    this.intentValidator = new IntentValidator();
    this.queryBuilder = new QueryBuilder();
  }

  async initialize(): Promise<void> {
    await this.schemaContext.initialize();
  }

  async search(
    userQuery: string,
```

```
  callerProfileId: number,
  accountId: number
): Promise<IAISearchResult> {
  const startTime = Date.now();

  try {
    // Step 1: Call AI provider
    const aiResponse = await this.provider.parseSearchIntent(
      userQuery,
      this.schemaContext.getSystemPrompt()
    );

    // Step 2: Validate AI output
    const intent = this.intentValidator.validate(aiResponse.intent);

    // Step 3: Check confidence threshold
    if (intent.confidence < 0.3) {
      return {
        success: false,
        message: 'Could not understand the search query. Please try rephrasing.',
        data: {
          profiles: [],
          interpretation: intent.interpretation,
          filters_applied: intent.filters,
          resolved_ids: {},
          confidence: intent.confidence,
          result_count: 0,
          ai_provider: this.provider.name,
          ai_model: this.provider.model,
          tokens_used: aiResponse.tokensUsed.total,
          response_time_ms: Date.now() - startTime,
        },
      };
    }

    // Step 4: Resolve lookup text → IDs
    const resolvedIds = this.lookupResolver.resolve(
      intent.filters,
      this.schemaContext.getLookupCache()
    );

    // Step 5: Build parameterized SQL
    const { sql, params } = this.queryBuilder.build(
      intent,
      resolvedIds,
      callerProfileId
    );

    // Step 6: Execute query
    const [rows] = await pool.execute(sql, params);
    const profiles = (rows as any[]) || [];
```

```
    // Step 7: Log the search (fire-and-forget)
    const totalTimeMs = Date.now() - startTime;
    setImmediate(() => {
      this.logSearch(
        accountId,
        callerProfileId,
        userQuery,
        intent,
        resolvedIds,
        sql,
        profiles.length,
        this.provider.name,
        this.provider.model,
        aiResponse.tokensUsed.total,
        totalTimeMs,
        intent.confidence,
        null
      );
    });

    return {
      success: true,
      data: {
        profiles,
        interpretation: intent.interpretation,
        filters_applied: intent.filters,
        resolved_ids: resolvedIds,
        confidence: intent.confidence,
        result_count: profiles.length,
        ai_provider: this.provider.name,
        ai_model: this.provider.model,
        tokens_used: aiResponse.tokensUsed.total,
        response_time_ms: totalTimeMs,
      },
    };
  } catch (error: any) {
    const totalTimeMs = Date.now() - startTime;

    // Log failed search
    setImmediate(() => {
      this.logSearch(
        accountId, callerProfileId, userQuery,
        null, null, null, 0,
        this.provider.name, this.provider.model,
        0, totalTimeMs, 0, error.message
      );
    });

    throw error;
  }
}
```

```
  private async logSearch(...args: any[]): Promise<void> {
    try {
      await pool.execute(
        'CALL eb_ai_search_log_create(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)',
        args
      );
    } catch (err) {
      console.error('Failed to log AI search:', err);
    }
  }
}
```

---

8. API Endpoints

8.1 AI Search

```
POST /api/ai-search
```

Auth: JWT + API Key + Premium subscription check

Request Body:

```
{
  "query": "Find Hindu women between 25-30, MBA, working in IT, from Hyderabad, vegetarian",
  "profile_id": 42
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "profiles": [
      {
        "profile_id": 87,
        "first_name": "Priya",
        "last_name": "Sharma",
        "gender": 2,
        "birth_date": "1997-05-14",
        "age": 28,
        "religion": 3,
        "marital_status": 1,
        "height_cms": 165,
        "profession": 12,
        "short_summary": "Software engineer at TCS...",
        "primary_photo_url": "https://..."
      }
    ],
    "interpretation": "Searching for Hindu females, age 25-30, with MBA education, in IT
sector, located in Hyderabad, vegetarian diet",
```

```
    "filters_applied": {
      "gender": "Female",
      "religion": "Hindu",
      "min_age": 25,
      "max_age": 30,
      "education_level": "MBA",
      "occupation": "IT",
      "city": "Hyderabad",
      "eating_habit": "Vegetarian"
    },
    "resolved_ids": {
      "gender": 2,
      "religion": 3,
      "education_level": 12,
      "occupation": 45,
      "eating_habit": 1
    },
    "confidence": 0.92,
    "result_count": 12,
    "ai_provider": "openai",
    "ai_model": "gpt-4o-mini",
    "tokens_used": 285,
    "response_time_ms": 1840
  }
}
```

Error Responses:

Status	Condition
400	Missing/empty query or profile_id
401	Invalid or missing authentication
403	User not on premium plan or quota exceeded
422	AI could not interpret the query (confidence < 0.3)
429	Rate limit exceeded
500	AI provider error or database error
504	AI provider timeout

8.2 Search History

```
GET /api/ai-search/history?limit=20
```

Auth: JWT + API Key

Response:

```
{
  "success": true,
  "data": [
    {
      "search_id": 145,
      "user_query": "Hindu women, 25-30, MBA, Hyderabad",
      "interpretation": "...",
      "result_count": 12,
      "confidence": 0.92,
      "created_at": "2026-03-16T10:30:00Z"
    }
  ]
}
```

8.3 Search Refinement

```
POST /api/ai-search/refine
```

Auth: JWT + API Key + Premium

Request Body:

```
{
  "previous_search_id": 145,
  "refinement": "same but age 22-28 and add Bangalore also",
  "profile_id": 42
}
```

The service loads the previous search's filters, sends both the original filters and the refinement text to the AI, and returns an updated result set.

8.4 Cache Management

```
POST /api/ai-search/refresh-cache
```

Auth: Admin only

Reloads all lookup values from the database into the in-memory schema context cache.

---

9. Database Changes

9.1 New Tables

`ai_search_log` **— Audit trail for every AI search**

```
CREATE TABLE ai_search_log (
  search_id         INT AUTO_INCREMENT PRIMARY KEY,
  account_id        INT NOT NULL,
  profile_id        INT NOT NULL,
```

```
  user_query        TEXT NOT NULL,
  interpreted_query JSON NULL,
  resolved_filters  JSON NULL,
  sql_executed      TEXT NULL,
  result_count      INT DEFAULT 0,
  ai_provider       VARCHAR(50) NULL,
  ai_model          VARCHAR(100) NULL,
  tokens_used       INT DEFAULT 0,
  response_time_ms  INT DEFAULT 0,
  confidence        DECIMAL(3,2) DEFAULT 0.00,
  error_message     TEXT NULL,
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_aisearch_account   (account_id),
  INDEX idx_aisearch_profile   (profile_id),
  INDEX idx_aisearch_created   (created_at),
  INDEX idx_aisearch_provider  (ai_provider)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

`ai_search_quota` **— Premium quota tracking**

```
CREATE TABLE ai_search_quota (
  quota_id          INT AUTO_INCREMENT PRIMARY KEY,
  account_id        INT NOT NULL,
  searches_used     INT DEFAULT 0,
  searches_limit    INT DEFAULT 5,
  period_start      DATE NOT NULL,
  period_end        DATE NOT NULL,
  plan_type         ENUM('free','basic','premium','unlimited') DEFAULT 'free',
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  UNIQUE INDEX idx_aiquota_account (account_id),
  INDEX idx_aiquota_period (period_start, period_end)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9.2 New Stored Procedures

Stored Procedure	Parameters	Purpose
`eb_ai_search_log_create`	account_id, profile_id, user_query, interpreted_query, resolved_filters, sql_executed, result_count, ai_provider, ai_model, tokens_used, response_time_ms, confidence, error_message	Insert a search audit log entry
`eb_ai_search_history_get`	account_id, limit_count	Retrieve a user's past AI searches
`eb_ai_search_quota_get`	account_id	Get current quota usage and limits

`eb_ai_search_quota_increment`	account_id	Increment searches_used by 1
`eb_ai_search_quota_reset`	account_id, new_limit, plan_type	Reset quota for new billing period

---

10. Security & Safety

Defense-in-Depth Layers

Layer	Mechanism	What it Prevents
Authentication	JWT token (HttpOnly cookie)	Unauthenticated access
API Key	`X-API-Key` header validation	Unknown client access
Premium Gate	Subscription status check	Non-paying users accessing premium feature
Quota Enforcement	`ai_search_quota` table	Cost overrun / abuse
Rate Limiting	Express rate limiter (5 req/min)	Burst abuse
Input Sanitization	Trim, max length 500 chars	XSS / oversized input
AI Output Validation	IntentValidator JSON schema check	Malformed / malicious AI responses
Lookup Whitelist	LookupResolver only matches known IDs	Invalid filter values
Parameterized SQL	`?` placeholders for every value	SQL injection
Column Whitelist	QueryBuilder uses only hardcoded columns	Schema leakage
Result Cap	`LIMIT 50` on every query	Full-table scan / memory overuse
AI Timeout	15-second timeout on AI API call	Hung connections
Prompt Injection Guard	System prompt instructs JSON-only output	Prompt injection attempts
Audit Logging	Every search logged to `ai_search_log`	Forensics / debugging

Prompt Injection Mitigation

The AI system prompt includes explicit instructions to:

1. Return ONLY valid JSON — no markdown, no code blocks
2. Never include SQL or executable code
3. If the query is unrelated to profile search, return empty filters
4. The AI response is always parsed as JSON and validated against a strict schema
5. Only validated, sanitized values are used in the parameterized query

Even if an attacker injects malicious text into the search query, the worst case is:

- AI returns garbage JSON → validator rejects → error response
- AI returns valid but nonsensical filters → query returns 0 results
- **No SQL injection possible** because the AI never generates SQL

---

11. Premium Gating & Billing

Subscription Tiers

Plan	AI Searches / Month	Price (suggested)
Free	5	$0
Basic	50	Part of existing basic plan
Premium	500	Part of existing premium plan
Unlimited	Unlimited	Enterprise/custom

Integration with Stripe

The existing Stripe checkout flow (`POST /api/stripe/create-session`) can be extended:

1. When a payment succeeds (via webhook), update `ai_search_quota` with the new plan's limits
2. The `premiumCheck` middleware reads the quota table to verify:
   - Account has an active plan
   - `searches_used < searches_limit`
   - Current date is within `period_start` — `period_end`
3. After a successful search, `searches_used` is incremented

Premium Check Middleware

```
// src/middlewares/premiumCheck.middleware.ts

export const checkAISearchQuota = async (
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
) => {
  try {
    const accountId = req.user?.account_id;

    const [rows] = await pool.execute(
      'CALL eb_ai_search_quota_get(?)',
      [accountId]
    );

    const quota = (rows as any[])[0]?.[0];

    if (!quota) {
      return res.status(403).json({
```

```
        success: false,
        message: 'AI search requires a subscription plan',
        error_code: 'AI_SEARCH_NO_PLAN'
      });
    }

    if (quota.searches_used >= quota.searches_limit) {
      return res.status(403).json({
        success: false,
        message: `Monthly AI search limit reached (${quota.searches_limit}). Upgrade your
plan for more searches.`,
        error_code: 'AI_SEARCH_QUOTA_EXCEEDED',
        data: {
          searches_used: quota.searches_used,
          searches_limit: quota.searches_limit,
          plan_type: quota.plan_type,
          period_end: quota.period_end
        }
      });
    }

    // Attach quota info to request for downstream use
    (req as any).aiQuota = quota;
    next();
  } catch (error) {
    console.error('AI quota check error:', error);
    return res.status(500).json({
      success: false,
      message: 'Error checking AI search quota'
    });
  }
};
```

---

12. File Structure

```
src/
├── controllers/
│   └── aiSearch.controller.ts            ← Request handling, validation
│
├── routes/
│   └── aiSearch.routes.ts                ← Route definitions + Swagger docs
│
├── interfaces/
│   └── aiSearch.interface.ts             ← TypeScript types
│
├── middlewares/
│   └── premiumCheck.middleware.ts         ← Premium subscription gate
│
├── services/
│   └── aiSearch/
```

```
│       ├── providers/
│       │   ├── aiProvider.interface.ts    ← IAIProvider interface
│       │   ├── openai.provider.ts        ← OpenAI GPT implementation
│       │   ├── gemini.provider.ts        ← Google Gemini implementation
│       │   ├── claude.provider.ts        ← Anthropic Claude implementation
│       │   └── deepseek.provider.ts      ← DeepSeek implementation
│       │
│       ├── aiProviderFactory.ts          ← Provider factory function
│       ├── aiSearch.service.ts           ← Main orchestrator
│       ├── schemaContext.ts              ← Prompt builder + lookup cache
│       ├── intentValidator.ts            ← AI output JSON validation
│       ├── queryBuilder.ts               ← Intent → parameterized SQL
│       └── lookupResolver.ts             ← Text → lookup ID resolution
│
└── index.ts                              ← Register /api/ai-search routes
```

New npm dependencies:

```
{
  "openai": "^4.x",
  "@google/generative-ai": "^0.x",
  "@anthropic-ai/sdk": "^0.x"
}
```

Only the provider you configure via `AI_PROVIDER` is called at runtime. All SDK packages are optional — install only the one you use.

---

13. Implementation Phases

Phase 1 — Core MVP (~3-4 days)

Task	Estimated
`aiSearch.interface.ts` — all TypeScript types	2 hours
`aiProvider.interface.ts` — provider contract	1 hour
`openai.provider.ts` — OpenAI implementation	3 hours
`schemaContext.ts` — prompt builder + lookup cache	4 hours
`intentValidator.ts` — JSON schema validation	2 hours
`lookupResolver.ts` — fuzzy text → ID matching	3 hours
`queryBuilder.ts` — parameterized SQL builder	4 hours
`aiSearch.service.ts` — orchestrator	4 hours
`aiSearch.controller.ts` — request handling	2 hours
`aiSearch.routes.ts` — routes + Swagger	2 hours

`ai_search_log` table + SP	2 hours
Integration in `index.ts`	0.5 hours
Basic rate limiting	1 hour
Testing (unit + integration)	4 hours

Phase 2 — Premium Gating (~1-2 days)

Task	Estimated
`ai_search_quota` table + SPs	2 hours
`premiumCheck.middleware.ts`	2 hours
Stripe webhook integration for plan updates	3 hours
Quota enforcement in search flow	2 hours
Testing	2 hours

Phase 3 — Additional Providers (~1-2 days)

Task	Estimated
`gemini.provider.ts`	2 hours
`deepseek.provider.ts`	2 hours
`claude.provider.ts`	2 hours
Provider fallback chain (if primary fails, try secondary)	3 hours
Testing across providers	2 hours

Phase 4 — Search History & Refinement (~1-2 days)

Task	Estimated
`eb_ai_search_history_get` SP	1 hour
History endpoint in controller/routes	2 hours
Search refinement endpoint	3 hours
Saved searches feature	2 hours
Testing	2 hours

Phase 5 — Monitoring & Optimization (~1 day)

Task	Estimated
Token usage tracking dashboard	2 hours

Provider latency monitoring	2 hours
Prompt optimization based on logged queries	2 hours
Cache TTL and auto-refresh	1 hour

Total estimated effort: 8–11 days

---

14. End-to-End Example

User Input

```
"I'm looking for a tall, well-educated Brahmin girl from Mumbai
who is vegetarian and doesn't drink"
```

Step 1: Controller Validation

- ✅ JWT valid, API key valid
- ✅ Premium plan active, quota: 3/50 used
- ✅ Query length: 98 chars (< 500 limit)

Step 2: AI Provider Call

The system sends to OpenAI:

```
System: [schema context with all lookup values]
User: "I'm looking for a tall, well-educated Brahmin girl from Mumbai
who is vegetarian and doesn't drink"
```

Step 3: AI Response

```
{
  "filters": {
    "gender": "Female",
    "religion": "Hindu",
    "caste": "Brahmin",
    "min_height_cms": 170,
    "education_level": "Post Graduate",
    "city": "Mumbai",
    "eating_habit": "Vegetarian",
    "drinking": "Non-Drinker"
  },
  "sort_by": "newest",
  "confidence": 0.88,
  "interpretation": "Looking for tall Brahmin Hindu women in Mumbai with post-graduate
education, vegetarian diet, non-drinker"
}
```

Step 4: Intent Validation

- ✅ JSON structure valid

- ✅ All filter types correct
- ✅ Confidence 0.88 > 0.3 threshold
- ✅ All values within allowed ranges

Step 5: Lookup Resolution

Filter	Text	Resolved ID
gender	"Female"	2
religion	"Hindu"	3
caste	"Brahmin"	42
education_level	"Post Graduate"	8
eating_habit	"Vegetarian"	1
drinking	"Non-Drinker"	1

(IDs are examples — actual values depend on your lookup_table data)

Step 6: Query Builder Output

```
SELECT DISTINCT
  pp.profile_id, pp.first_name, pp.last_name, pp.gender,
  pp.birth_date, TIMESTAMPDIFF(YEAR, pp.birth_date, CURDATE()) AS age,
  pp.religion, pp.caste, pp.marital_status, pp.height_cms,
  pp.profession, pp.short_summary,
  ph.url AS primary_photo_url
FROM profile_personal pp
  LEFT JOIN profile_address pa ON pp.profile_id = pa.profile_id AND pa.softdelete = 0
  LEFT JOIN profile_education pe ON pp.profile_id = pe.profile_id AND pe.softdelete = 0
  LEFT JOIN profile_lifestyle pl ON pp.profile_id = pl.profile_id AND pl.softdelete = 0
  LEFT JOIN profile_photo ph ON pp.profile_id = ph.profile_id AND ph.photo_type = 450 AND
ph.softdelete = 0
WHERE pp.softdelete = 0
  AND pp.profile_id != ?       -- [42]
  AND pp.gender = ?            -- [2]
  AND pp.religion = ?          -- [3]
  AND pp.caste = ?             -- [42]
  AND pp.height_cms >= ?       -- [170]
  AND pa.city LIKE ?           -- ['%Mumbai%']
  AND pe.education_level = ?   -- [8]
  AND pl.eating_habit = ?      -- [1]
  AND pl.drink_frequency = ?   -- [1]
ORDER BY pp.created_date DESC
LIMIT 50

-- Params: [42, 2, 3, 42, 170, '%Mumbai%', 8, 1, 1]
```

Step 7: Results

MySQL returns 7 matching profiles.

Step 8: Response

```
{
  "success": true,
  "data": {
    "profiles": [/* 7 profiles */],
    "interpretation": "Looking for tall Brahmin Hindu women in Mumbai with post-graduate
education, vegetarian diet, non-drinker",
    "filters_applied": {
      "gender": "Female",
      "religion": "Hindu",
      "caste": "Brahmin",
      "min_height_cms": 170,
      "education_level": "Post Graduate",
      "city": "Mumbai",
      "eating_habit": "Vegetarian",
      "drinking": "Non-Drinker"
    },
    "confidence": 0.88,
    "result_count": 7,
    "ai_provider": "openai",
    "ai_model": "gpt-4o-mini",
    "tokens_used": 285,
    "response_time_ms": 1840
  }
}
```

---

15. Error Handling

Error Scenario	Response	Status
Empty or missing query	`{ message: "Search query is required" }`	400
Query too long (> 500 chars)	`{ message: "Query must be under 500 characters" }`	400
No premium plan	`{ message: "AI search requires a subscription", error_code: "AI_SEARCH_NO_PLAN" }`	403
Quota exceeded	`{ message: "Monthly limit reached", error_code: "AI_SEARCH_QUOTA_EXCEEDED" }`	403
Rate limited	`{ message: "Too many requests" }`	429
AI returns unparseable response	`{ message: "AI service returned invalid response. Please try again." }`	502
AI confidence too low (< 0.3)	`{ message: "Could not understand query. Please rephrase." }`	422
AI provider timeout (> 15s)	`{ message: "AI service timed out. Please try again." }`	504

AI provider API error	`{ message: "AI service temporarily unavailable" }`	503
Database query error	`{ message: "Search failed. Please try again." }`	500
Lookup resolution fails (0 IDs resolved)	`{ message: "Could not match any filters. Try different terms." }`	422

---

16. Performance Considerations

Latency Budget

Component	Expected Latency
Auth + API Key validation	~5ms
Premium/Quota check (DB)	~10ms
AI Provider API call	~1,000–2,500ms
Lookup resolution (in-memory)	~1ms
Query building	~1ms
MySQL query execution	~50–200ms
Logging (async, non-blocking)	0ms (fire-and-forget)
Total	**~1,100–2,800ms**

The AI API call dominates latency. This is acceptable for a premium feature.

Database Query Optimization

1. **Indexes**: Ensure the following indexes exist:
    - `profile_personal(gender, religion, caste, marital_status)`
    - `profile_personal(birth_date)`
    - `profile_address(city, state, country)`
    - `profile_education(education_level)`
    - `profile_employment(job_title_id)`
    - `profile_lifestyle(eating_habit, drink_frequency)`
2. **LIMIT 50**: Every AI search query is capped at 50 results
3. **Conditional JOINs**: The query builder only adds JOINs for tables actually needed by the filters — a search for "Hindu women 25-30" will NOT join education, employment, or lifestyle tables

Caching Strategy

Cache	TTL	Storage
Lookup values	Until manual refresh	In-memory (SchemaContext)
AI system prompt	Rebuilt on lookup refresh	In-memory

Search results	Not cached (always fresh)	—
Quota check	Per-request (no cache)	—

---

17. Monitoring & Observability

Key Metrics to Track

Metric	Source	Purpose
Searches per hour/day	`ai_search_log`	Usage trends
Avg response time	`ai_search_log.response_time_ms`	Performance monitoring
Avg tokens per search	`ai_search_log.tokens_used`	Cost tracking
Confidence distribution	`ai_search_log.confidence`	Prompt quality
Error rate	`ai_search_log.error_message IS NOT NULL`	Reliability
Zero-result rate	`ai_search_log.result_count = 0`	Query/data gap analysis
Provider breakdown	`ai_search_log.ai_provider`	Provider comparison
Top search terms	`ai_search_log.user_query` (aggregate)	Feature insights
Quota utilization	`ai_search_quota`	Plan sizing

Alerting Thresholds (suggested)

Alert	Condition
High error rate	> 10% of searches fail in 1 hour
AI provider down	> 5 consecutive timeouts
Cost spike	> 2x normal daily token usage
Zero-result spike	> 50% of searches return 0 results

---

18. Future Enhancements

Short-term (Post-MVP)

Enhancement	Description
Conversational refinement	"Same search but change age to 22-28"
Saved searches	Save and re-run frequent AI searches
Search suggestions	Show popular/trending search queries
Multi-language support	Accept queries in Hindi, Telugu, Tamil, etc.

Medium-term

Enhancement	Description
Semantic matching	Use embeddings to match "software developer" with "programmer", "coder", etc.
AI-powered recommendations	"Based on your profile, you might like..."
Voice search	Speech-to-text → AI search pipeline
Search analytics dashboard	Admin view of search patterns and popular criteria

Long-term

Enhancement	Description
Fine-tuned model	Train a small model specifically for matrimony search intent parsing
Vector database	Store profile embeddings for semantic similarity search
Hybrid search	Combine structured filters with vector similarity for best results
Auto-suggestions	As user types, show real-time AI-powered completions

---

19. Appendix

A. Sample AI Provider Implementation (OpenAI)

```
// src/services/aiSearch/providers/openai.provider.ts

import OpenAI from 'openai';
import { IAIProvider, IAISearchResponse } from './aiProvider.interface';
import { ISearchIntent } from '../../../interfaces/aiSearch.interface';

export class OpenAIProvider implements IAIProvider {
  name = 'openai';
  model = process.env.AI_MODEL || 'gpt-4o-mini';

  private client: OpenAI;

  constructor() {
    this.client = new OpenAI({
      apiKey: process.env.AI_API_KEY,
    });
  }

  async parseSearchIntent(
    userQuery: string,
    schemaContext: string
  ): Promise<IAISearchResponse> {
```

```
    const startTime = Date.now();

    const response = await this.client.chat.completions.create({
      model: this.model,
      messages: [
        { role: 'system', content: schemaContext },
        { role: 'user', content: userQuery },
      ],
      temperature: parseFloat(process.env.AI_TEMPERATURE || '0.1'),
      max_tokens: parseInt(process.env.AI_MAX_TOKENS || '500'),
      response_format: { type: 'json_object' },
    });

    const content = response.choices[0]?.message?.content || '{}';
    const intent: ISearchIntent = JSON.parse(content);

    return {
      intent,
      tokensUsed: {
        prompt: response.usage?.prompt_tokens || 0,
        completion: response.usage?.completion_tokens || 0,
        total: response.usage?.total_tokens || 0,
      },
      responseTimeMs: Date.now() - startTime,
    };
  }
}
```

B. Sample AI Provider Implementation (Gemini)

```
// src/services/aiSearch/providers/gemini.provider.ts

import { GoogleGenerativeAI } from '@google/generative-ai';
import { IAIProvider, IAISearchResponse } from './aiProvider.interface';

export class GeminiProvider implements IAIProvider {
  name = 'gemini';
  model = process.env.AI_MODEL || 'gemini-2.0-flash';

  private client: GoogleGenerativeAI;

  constructor() {
    this.client = new GoogleGenerativeAI(process.env.AI_API_KEY || '');
  }

  async parseSearchIntent(
    userQuery: string,
    schemaContext: string
  ): Promise<IAISearchResponse> {
    const startTime = Date.now();
```

```
    const model = this.client.getGenerativeModel({
      model: this.model,
      generationConfig: {
        temperature: parseFloat(process.env.AI_TEMPERATURE || '0.1'),
        maxOutputTokens: parseInt(process.env.AI_MAX_TOKENS || '500'),
        responseMimeType: 'application/json',
      },
    });

    const result = await model.generateContent([
      { role: 'user', parts: [{ text: `${schemaContext}\n\nUser query: ${userQuery}` }] },
    ]);

    const content = result.response.text();
    const intent = JSON.parse(content);

    const usage = result.response.usageMetadata;

    return {
      intent,
      tokensUsed: {
        prompt: usage?.promptTokenCount || 0,
        completion: usage?.candidatesTokenCount || 0,
        total: usage?.totalTokenCount || 0,
      },
      responseTimeMs: Date.now() - startTime,
    };
  }
}
```

C. Complete Route Registration

```
// src/index.ts — add to existing routes

import aiSearchRoutes from './routes/aiSearch.routes';

// After other route registrations:
app.use('/api/ai-search', aiSearchRoutes);
```

D. Glossary

Term	Definition
Search Intent	Structured JSON representation of what the user is looking for
Schema Context	The system prompt sent to the AI, containing available filters and valid values
Lookup Resolution	Converting text values (e.g., "Hindu") to numeric IDs (e.g., 3)
Query Builder	Component that converts validated intent into parameterized SQL

Provider Factory	Design pattern that creates the appropriate AI provider based on configuration
Quota	The number of AI searches allowed per billing period
Confidence	AI's self-assessed certainty (0.0–1.0) in its interpretation
Intent Validator	Component that checks AI output against the expected JSON schema

---

> ***Document Version:*** *1.0* ***Author:*** *AI Engineering Team* ***Review Status:*** *Pending Review* ***Next Steps:*** *Review and approve architecture, then begin Phase 1 implementation*